OFFICIAL VERISIGHT OPERATIONAL JOURNAL & FIELD ENGINEERING MANUAL

VERISIGHT™

Autonomous Silicon Verification & Failure Debugging Copilot

Dual-Engine Architecture: Diagnostic Debug RCA & Verification Plan Synthesizer

Document Title:	VeriSight Comprehensive Operator's Journal & Technical Reference
Lead Author & Architect:	**Patel Kittu** (GitHub: *@KITTUV03*)
Engineering Affiliation:	Mirafra Technologies • Silicon Systems & AI Verification Group
Edition & Version:	Production Release v2.4 (Enterprise Dual-Engine Suite)
Release Date:	October 20, 2026
Core Capabilities:	• Autonomous Simulation Failure Debugging (AST, CDC, Lint, X-Tracer) • Verification Plan, Functional Coverage & SVA Synthesis • ISO 26262 / DO-254 Bi-Directional Traceability Matrix • Interactive Guided Wizard & Headless CI/CD Regression Modes
Supported Standards:	IEEE 1800 SystemVerilog, UVM 1.2, SVA (Assertions), VCD/FSDB, PDF/Markdown Specs

CONFIDENTIAL & PROPRIETARY: This technical operational journal is prepared for semiconductor verification engineers, system architects, and design verification leads. No part of this document or software architecture may be reproduced without prior authorization from Patel Kittu.

Document Overview & Table of Contents

Chapter / Section	Core Subject Matter & Operational Focus	Key Artifacts / Screenshots
Chapter 1	Introduction to VeriSight & The Autonomous Verification Paradigm	System Philosophy, Architecture
Chapter 2	Quick Start, Installation & Interactive Setup Wizard	Screenshot 01 (Welcome Menu)
Chapter 3	Step 2: LLM Configuration & Deterministic Fallback Mode	Screenshot 02 (LLM Configuration)
Chapter 4	VeriSight Agent 1: Root-Cause Failure Debugging (RCA) Engine	Screenshots 03 & 04 (Debug Inputs & Results)
Chapter 5	VeriSight Agent 2: Verification Plan & Coverage/SVA Architect	Screenshots 05 & 06 (Plan Inputs & Results)
Chapter 6	Option 3: Pre-Packaged Instant Demonstration Suite	Screenshot 07 (Quick Demos Menu)
Chapter 7	Option 4: Clean Session Termination & Advanced Headless CLI	Screenshot 08 (Exit & CLI Examples)
Chapter 8	Verification Engineering Best Practices & Sign-Off Checklist	ISO 26262 / DO-254 Compliance
Chapter 9	Schema Reference, Configuration Variables & Troubleshooting	Error Codes & Remediation
Appendix	Operator Quick-Reference Cheatsheet & Acronym Glossary	CLI Command Dictionary

Executive Summary from Author: Patel Kittu

Modern Very Large Scale Integration (VLSI) and System-on-Chip (SoC) verification cycles frequently consume between **60% and 75% of total ASIC project development time**. Functional verification teams spend thousands of engineering hours navigating gigabyte-scale simulation logs, tracing elusive multi-clock asynchronous Clock Domain Crossing (CDC) faults, resolving Unknown State (X) propagation bugs, and authoring exhaustive functional coverage models and SystemVerilog Assertions (SVA) from hundreds of pages of textual design specifications.

VeriSight was engineered by **Patel Kittu** to revolutionize silicon verification through a coordinated multi-agent artificial intelligence framework. Rather than acting as a simple generic chat assistant, VeriSight is deeply integrated into EDA toolchains. It combines formal static analysis (AST parsers, syntax checkers, linting, connectivity graphs, netlist X-tracing) with cutting-edge Large Language Models (LLMs) including Google Gemini 2.0 Flash and Anthropic Claude 3.5 Sonnet, paired with a vectorized ChromaDB historical sign-off knowledge base.

This journal serves as the definitive reference manual for operating both primary engines of the VeriSight platform: **Agent 1 (VeriSight Debug)** for automated root-cause localization and patch synthesis, and **Agent 2 (VeriSight Plan)** for automated extraction of verification features, test scenarios, functional coverage groups, and SVA suites.

NOTE: VeriSight supports interactive guided operation via terminal UI (TUI) as well as non-blocking headless execution designed for continuous integration (CI/CD) regression farms.

Chapter 1: The VeriSight Architecture & Philosophy

VeriSight is structured around a decoupled, modular dual-engine architecture. Each engine operates autonomously or in tandem depending on the verification phase: early-stage architecture planning versus late-stage regression debugging.

Subsystem Engine	Target Verification Phase	Core Automated Responsibility
Agent 1: VeriSight Debug *(Autonomous RCA Engine)*	Simulation Regressions, Silicon Bring-up, Testbench Failures	• Parses UVM/VMM simulation logs & fatal assertions. • Extracts RTL AST, port connectivity, and netlists. • Executes deterministic X-Tracer gate-level backward propagation. • Runs rule-based Lint, CDC, and Protocol analyzers. • Synthesizes root-cause classification & generates verified git diff patches.
Agent 2: VeriSight Plan *(Autonomous Plan Architect)*	Pre-Silicon Spec Ingestion, Testbench Planning, Coverage Closure	• Ingests PDF, Markdown, and Text technical specifications. • Performs hierarchical feature decomposition & requirement tagging. • Formulates directed & constrained-random test scenarios. • Generates production SystemVerilog covergroups & cross bins. • Synthesizes SystemVerilog Assertions (SVA) sequences & properties. • Exports ISO 26262 / DO-254 Bi-Directional Traceability Matrices.
ChromaDB Vector Store *(RAG Knowledge Base)*	Continuous Cross-Project Learning & Sign-Off Memory	• Indexes historical post-silicon bugs, errata, and past verification plans. • Provides semantic retrieval of known silicon failure modes. • Prevents re-occurrence of legacy silicon defects.
Interactive Wizard *(Operator Interface)*	Developer Terminal & EDA Workstation	• Unified single-command launcher (./start.sh). • Auto-detects and validates project inputs with path fuzzy-matching. • Auto-launches interactive HTML verification dashboards in local browsers.

Verification Data Flow Pipeline

The VeriSight pipeline follows a strict five-stage deterministic sequence: **Ingestion** → **Static Analysis & AST Extraction** → **Semantic Knowledge Augmentation (RAG)** → **Multi-Agent LLM Synthesis** → **Artifact & Patch Generation**. By coupling deterministic parsers with LLM reasoning, VeriSight eliminates the risk of hallucination while preserving high cognitive flexibility.

Chapter 2: Installation, Environment & Wizard Initialization

VeriSight has been designed for frictionless deployment in standard Linux EDA workstation environments (CentOS, RHEL, Ubuntu) and Windows Subsystem for Linux (WSL2).

Prerequisites & Environment Setup

Ensure Python 3.10+ is available on your machine. Install VeriSight in editable mode within your project virtual environment:

```
# Clone the repository
git clone git@github.com:KITTUV03/VeriSight.git
cd VeriSight

# Activate your virtual environment and install dependencies
source .venv/bin/activate
pip install -e '.[dev]'
pip install pypdf reportlab
```

Launching the Interactive Guided Setup Wizard

To provide immediate usability without requiring engineers to remember complex CLI syntax, author **Patel Kittu** created the VeriSight Guided Wizard. The wizard can be launched using any of the following intuitive commands:

```
./start.sh # Native executable bash launcher
./start # Symlinked shortcut
python start.py # Python direct bootstrap
python main.py start # Main package entry point with 'start' verb
python main.py -i # Interactive flag
```

The VeriSight Main Welcome Screen

When launched, the operator is greeted by the VeriSight welcome banner and the **Step 1: Choose Autonomous Workflow** selection menu. The terminal screen presents four clearly demarcated operational pathways, as depicted below:

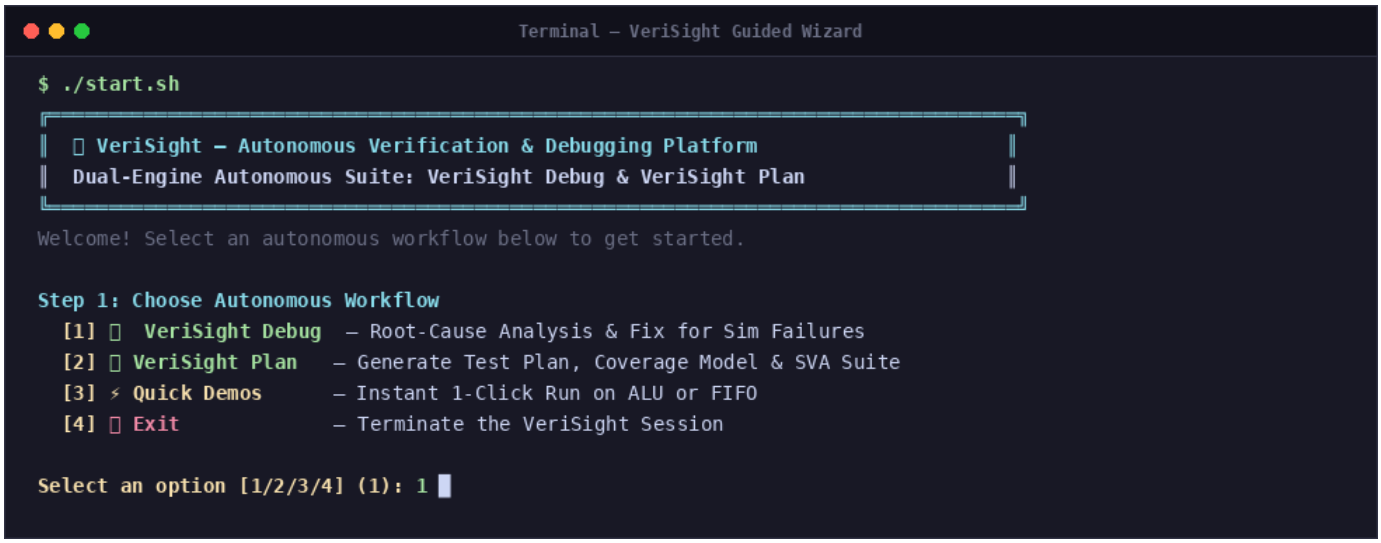


Figure 2.1: VeriSight Interactive Guided Setup Wizard — Main Welcome Banner & Step 1 Workflow Selection Menu

Detailed Breakdown of the 4 Workflow Choices:

- **[1] VeriSight Debug:** Initiates the autonomous Root-Cause Analysis (RCA) pipeline to diagnose simulation regressions, UVM testbench errors, and RTL bugs.
- **[2] VeriSight Plan:** Activates the Verification & Architecture Plan generator to ingest specification documents and synthesize test cases, functional coverage models, and SVA suites.
- **[3] Quick Demos:** Provides 1-click pre-packaged demonstrations on standard ALU and FIFO designs without requiring manual input typing.
- **[4] Exit:** Gracefully terminates the wizard session and displays advanced CLI examples for automation scripts.

Chapter 3: LLM Configuration & Deterministic Fallback Mode

VeriSight features an intelligent LLM configuration layer that automatically detects active API credentials from the host environment, offers seamless custom key configuration, and supports a 100% offline deterministic execution mode for secure air-gapped defense and automotive semiconductor environments.

Step 2: LLM Configuration Screen

Immediately after selecting an operational workflow in Step 1, VeriSight transitions to **Step 2: LLM Configuration**. The system scans the environment variables GEMINI_API_KEY and ANTHROPIC_API_KEY and displays an interactive status table:

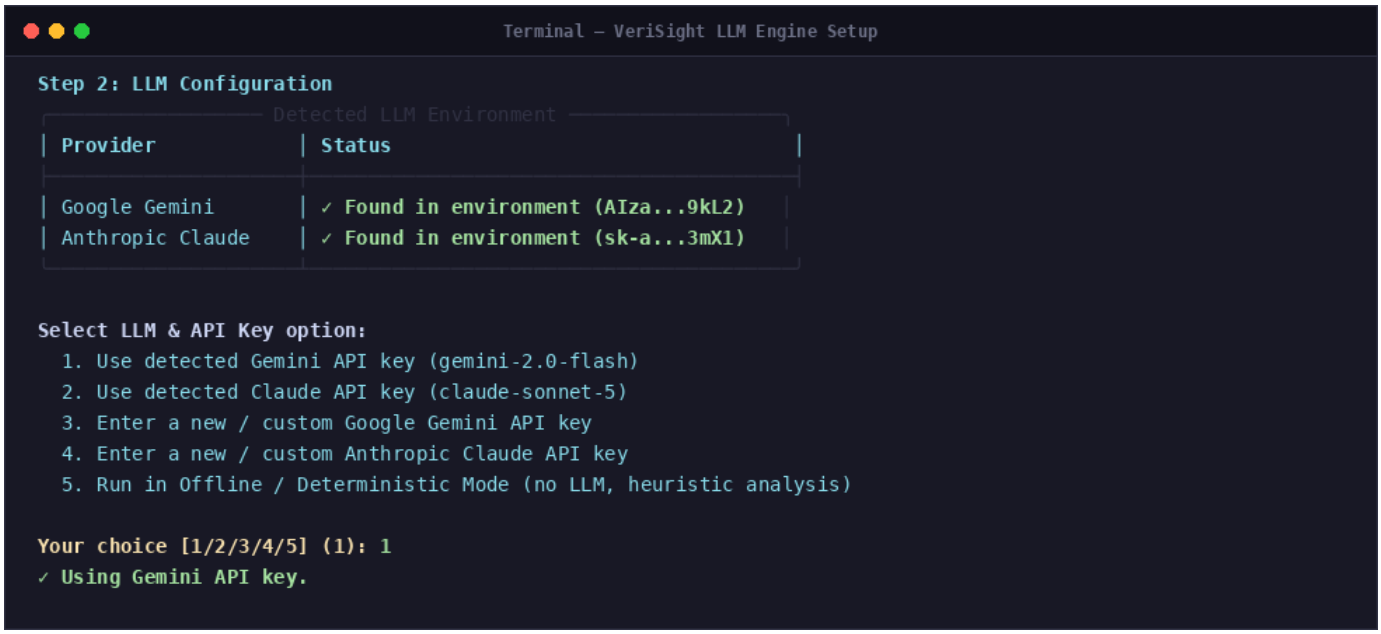


Figure 3.1: VeriSight LLM Configuration Screen — Multi-Provider Detection & Selection Table

Operational Modes Supported by the Engine:

Configuration Option	Target Engine / Model	Characteristics & Use-Case
1. Google Gemini (Detected)	gemini-2.0-flash *(Recommended Default)*	Ultra-fast latency, 1-million token context window, exceptional structured JSON schema compliance. Ideal for massive simulation logs and multi-page specifications.
2. Anthropic Claude (Detected)	claude-sonnet-5 / 3.5-sonnet	State-of-the-art HDL code reasoning, advanced temporal logic synthesis for complex SVA assertions, and high-fidelity Verilog patch generation.
3 & 4. Custom Key Entry	User-supplied API Key	Prompts the operator securely with masked input. VeriSight prompts whether to persist the entered key to the local .env file for zero-friction future sessions.
5. Offline Deterministic Mode	No Cloud LLM *(Deterministic Heuristics)*	Operates without external internet connectivity. Utilizes AST pattern matching, rule-based CDC analyzers, lint checkers, and netlist X-Tracers to produce verified diagnostics.

VERIFICATION TIP: In offline deterministic mode, VeriSight disables cloud LLM API calls and relies strictly on deterministic Python verification analyzers. This guarantees 100% data residency within confidential foundry and defense firewalls.

Chapter 4: VeriSight Agent 1 — Root-Cause Analysis & Fix Engine

VeriSight Agent 1 (VeriSight Debug) is an autonomous multi-modal root-cause analysis (RCA) pipeline. When modern chip simulations fail, debuggers are faced with millions of log lines, convoluted UVM transaction traces, and subtle sequential logic bugs. Agent 1 automates this entire diagnostic lifecycle.

Step-by-Step Operator Guide: Providing Debug Inputs

When Agent 1 is selected, the wizard guides the engineer through input specification. VeriSight incorporates intelligent path normalization (converting Windows drive letters like C:\ into WSL paths like /mnt/c/), extension auto-completion (automatically detecting .log, .sv, .md), and fuzzy directory searching:



Figure 4.1: Agent 1 Input Prompting — Simulation Log, RTL, Testbench, and Specification Ingestion

Explanation of Agent 1 Required & Optional Inputs:

- **Simulation Log (.log) [REQUIRED]:** The raw simulation log from Synopsys VCS, Cadence Xcelium, Siemens Questa, or Icarus Verilog containing UVM error banners, timestamps, and assertion failures.
- **RTL Source Directory (.sv, .v) [OPTIONAL]:** Design Under Test (DUT) hardware description files. VeriSight automatically discovers all RTL modules in the hierarchy.
- **UVM Testbench (.sv) [OPTIONAL]:** Testbench components (drivers, monitors, scoreboards, sequencers) for identifying testbench-level race conditions or scoreboard synchronization errors.
- **Design Specification (.md, .pdf) [OPTIONAL]:** Golden architectural specification to cross-verify whether the RTL implementation adheres to stated functional requirements.
- **Output Directory:** Destination for all diagnostic reports and generated code patches (defaults to output/custom_debug/).

The 5-Phase Diagnostic Execution Pipeline

Once inputs are confirmed, Agent 1 executes a synchronized 5-phase diagnostic sweep:

1. **Log & AST Parsing:** Tokenizes simulation logs, extracts UVM phases, simulation timestamps, and locates exact assertion failure points.

2. **Static Analyzer Suite:** Concurrently runs five specialized analyzers: Clock Domain Crossing (CDC), Linting (combinational loops, latches), Protocol Checking, Structural AST checks, and Gate-level X-Propagation Tracing.
3. **ChromaDB RAG Retrieval:** Queries the localized vector database for similar historical silicon bugs and validated resolution patterns.
4. **Multi-Agent LLM Synthesis:** Aggregates the static analyzer findings and queries the configured LLM with strict verification schemas to classify the bug and identify the exact module and line number.
5. **Fix Engine & Patch Generation:** Synthesizes a minimal, non-regressive SystemVerilog fix and outputs a standard Git patch file.

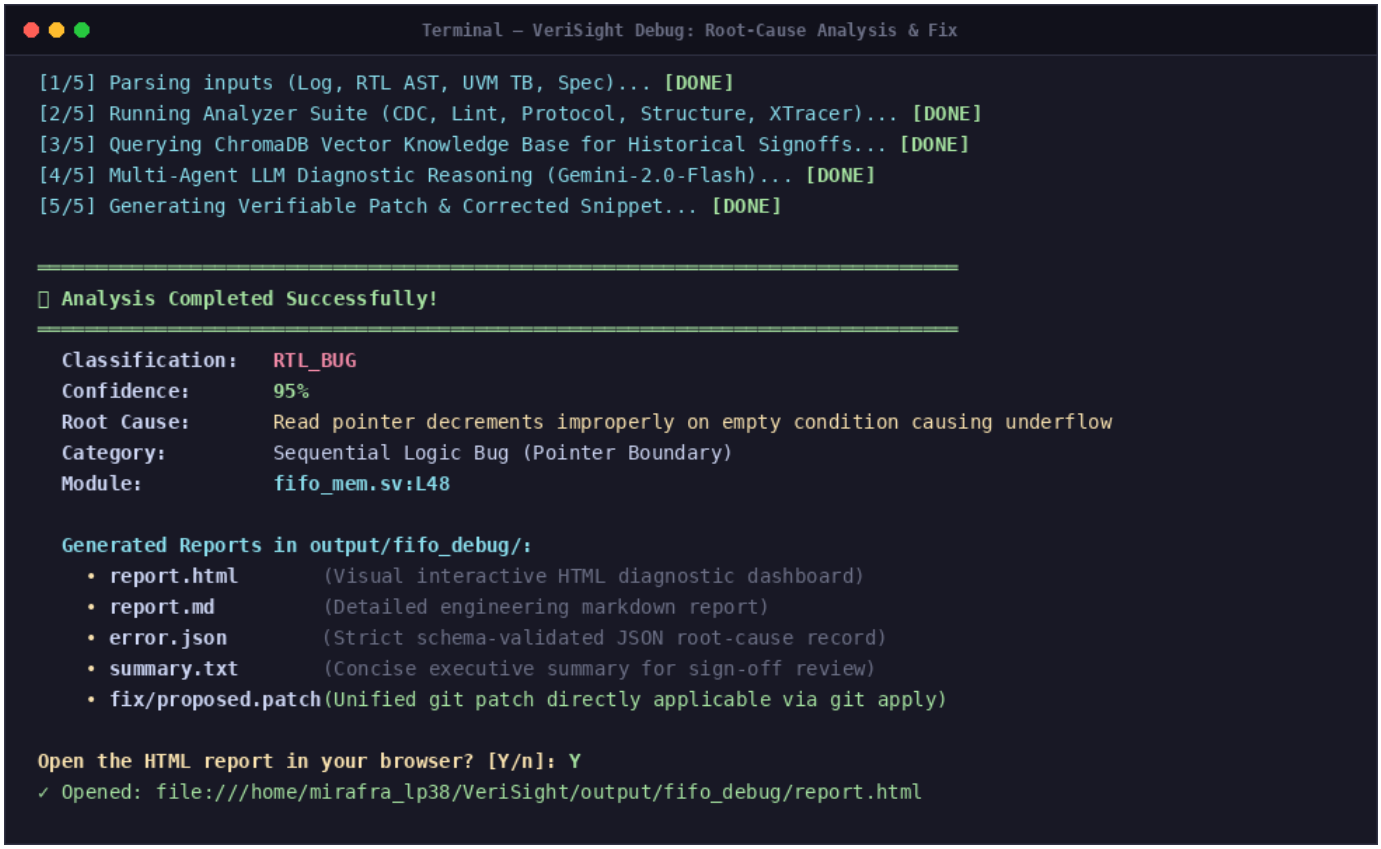


Figure 4.2: Agent 1 Execution Output & Results — Root Cause Classification, Confidence Score & Generated Reports

Output Artifacts Generated by Agent 1:

Generated Artifact	Format	Engineering Purpose & Value
report.html	Interactive HTML	Visual browser dashboard featuring collapsible error traces, confidence gauges, code diff viewers, and analyzer summaries. Auto-opens in the default web browser upon run completion.
report.md	Markdown	Comprehensive engineering documentation containing detailed diagnostic narrative, simulation timeline, and recommended post-fix verification tests. Ideal for Git commit logs.
error.json	JSON Schema	Machine-readable error descriptor containing classification (RTL_BUG, TB_BUG, ENV_BUG), confidence percentage, module name, and line numbers for automated CI/CD bug trackers (Jira, Bugzilla).
summary.txt	Plain Text	Concise executive briefing formatted for morning standups, verification status reports, and email notifications.
fix/proposed.patch	Git Diff Patch	Production-ready unified patch file. Can be directly previewed or applied to the repository via git apply output/fifo_debug/fix/proposed.patch.

Chapter 5: VeriSight Agent 2 — Verification Plan & Suite Architect

VeriSight Agent 2 (VeriSight Plan) represents a breakthrough in autonomous testbench architecture. In traditional silicon development, authoring a complete verification plan from architectural specifications takes weeks of manual work. Agent 2 ingests specifications in PDF, Markdown, or Text formats, analyzes the RTL interfaces, and synthesizes complete test plans, SystemVerilog functional coverage packages, and formal SVA assertion suites.

Step-by-Step Operator Guide: Providing Planning Inputs

From the main wizard menu, select option [2] VeriSight Plan. The wizard prompts for the specification document and optional RTL sources:

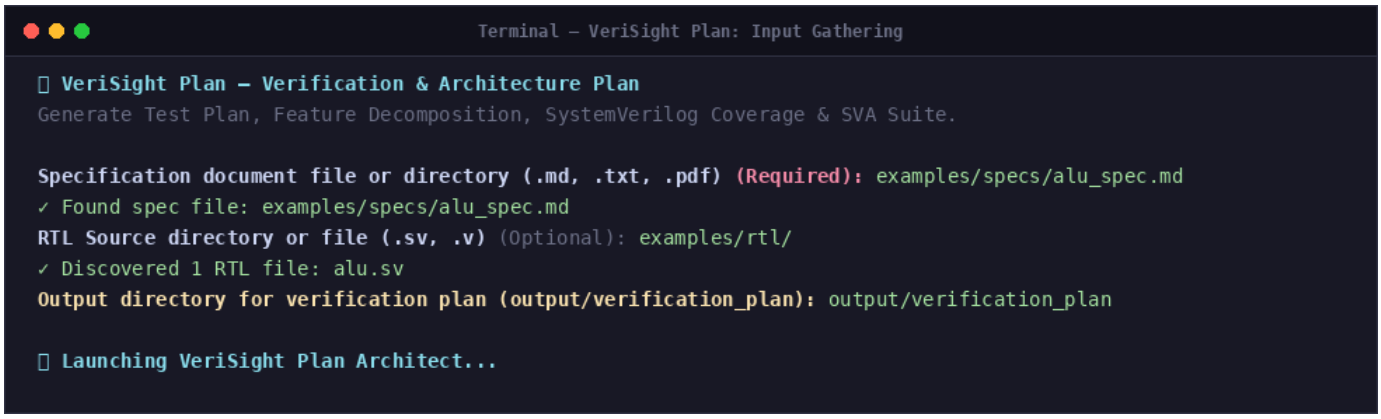


Figure 5.1: Agent 2 Input Prompting — Specification Document (PDF/MD) and RTL Ingestion

Key Inputs for Verification Planning:

- **Specification Document (.pdf, .md, .txt) [REQUIRED]:** The architectural specification describing device registers, protocol states, timing diagrams, and operational modes. Native PDF extraction is powered by pypdf.
- **RTL Source Directory (.sv, .v) [OPTIONAL]:** Top-level and sub-block RTL code. VeriSight parses port lists, clock/reset definitions, and parameter widths to ground the generated coverage and assertion models directly in actual hardware signal names.
- **Output Directory:** Target directory for the verification plan artifacts (defaults to output/verification_plan/).

The Autonomous Generation Architecture

Agent 2 executes five interconnected architectural modules to synthesize the complete verification suite:

1. **Feature Decomposition (feature_extractor.py):** Hierarchically decomposes the specification into discrete functional blocks (e.g., Arithmetic Operations, Status Flags, Overflow Handlers, Clocking, Reset Sequences).
2. **Test Planning (test_planner.py):** Synthesizes directed, constrained-random, corner-case, and error-injection test cases with explicit stimulus and expected checking criteria.
3. **Coverage Architecture (coverage_architect.py):** Formulates complete SystemVerilog covergroup structures, sampling clocks, coverpoints with custom bins, and multi-variable cross coverage constructs.
4. **Assertion Architecture (assertion_architect.py):** Formulates SystemVerilog Assertions (SVA) with temporal operators (|->, |=>, ##[1:5]) for protocol invariants and interface contracts.

5. **Traceability Matrix (traceability.py):** Constructs an ISO 26262 and DO-254 compliant bi-directional traceability graph mapping every specification requirement to corresponding RTL signals, test cases, coverage points, and SVA properties.

```
Terminal — VeriSight Plan: Generation Results

[1/6] Ingesting Spec Document (Markdown / Text / Multi-page PDF)... [DONE]
[2/6] Parsing RTL Port Interface & FSM States (alu.sv)... [DONE]
[3/6] Synthesizing Hierarchical Feature Tree & Test Matrix... [DONE]
[4/6] Generating SystemVerilog Functional Coverage Model (covergroup)... [DONE]
[5/6] Formulating SystemVerilog Assertions Suite (SVA Properties)... [DONE]
[6/6] Computing Bi-Directional Traceability Matrix & HTML Dashboard... [DONE]

═══════════════════════════════════════════════════════════════════════════
 Verification Plan Generated Successfully!
═══════════════════════════════════════════════════════════════════════════
  Target Design:            ALU_Top (Arithmetic Logic Unit)
  Decomposed Features:      8 verification functional units
  Defined Test Scenarios:   14 directed & constrained-random test cases
  Coverage Points:          12 covergroups & cross coverage bins
  SVA Formal Assertions:    10 temporal sequence & invariant properties

  Generated Artifacts in output/verification_plan/:
    • verification_plan.html  (Interactive HTML Dashboard with Live Filtering)
    • verification_plan.md    (Engineering Markdown Plan for git repository)
    • traceability_matrix.csv (ISO 26262/DO-254 Certified Bi-directional Matrix)
    • coverage_model.sv       (Production-ready SystemVerilog Covergroups)
    • dut_assertions.sv       (SystemVerilog Assertions Suite ready for TB bind)
    • verification_plan.json  (Full Schema-Validated JSON Architecture)

Open the interactive HTML Verification Plan in your browser? [Y/n]: Y
✓ Opened: file:///home/mirafra_lp38/VeriSight/output/verification_plan/verification_plan.html
```

Figure 5.2: Agent 2 Execution Output & Verification Plan Artifacts Generation

Artifacts Generated by Agent 2 in output/verification_plan/:

Artifact Filename	Language / Type	Content & Verification Functionality
verification_plan.html	HTML5 / Interactive	Executive verification dashboard with live search filtering, requirement coverage heatmaps, test status tracking, and expandable SVA code inspection.
verification_plan.md	Markdown	Comprehensive engineering verification plan formatted for direct inclusion in Git repositories, documentation wikis, and design review meetings.
traceability_matrix.csv	CSV Spreadsheet	Bi-directional traceability matrix satisfying ISO 26262 (ASIL-D) and DO-254 safety sign-off requirements: Requirement_ID → RTL_Signal → Test_ID → Covergroup → SVA_Property.
coverage_model.sv	SystemVerilog	Production-ready SystemVerilog functional coverage package containing parameterized covergroup models, explicit transition bins, and cross coverage ready for UVM subscriber integration.
dut_assertions.sv	SystemVerilog (SVA)	Synthesized SVA checker module ready for direct SystemVerilog bind instantiation onto the DUT top-level interface.
verification_plan.json	JSON Schema	Strict schema-compliant database containing all features, tests, coverage goals, and assertion definitions for programmatic ingestion by EDA coverage closure databases.

Chapter 6: Option 3 — Guided Quick Demonstration Suite

To allow new verification engineers, prospective users, and engineering leadership to evaluate VeriSight within seconds without manually preparing or browsing for input files, author **Patel Kittu** integrated the **Quick Demos Suite** into Option 3 of the setup wizard.

Selecting Pre-Packaged Demos

Selecting option [3] Quick Demos in Step 1 and confirming the LLM in Step 2 brings up the demo selection menu shown below:



Figure 6.1: Option 3 Quick Demos Menu — Instant 1-Click Verification & Debug Demos

Deep Dive into the 4 Pre-Configured Demonstrations:

Demo Option	Target Hardware & Bug Profile	Pre-Configured Inputs & Expected Outcome
[1] ALU Debug Demo	Arithmetic Logic Unit (ALU) *Combinational Latch / Missing Reset*	**Inputs:** examples/specs/alu_spec.md, examples/rtl/, examples/logs/sim.log. **Outcome:** Detects uninitialized accumulator output during reset and produces a patch adding synchronous reset clearing.
[2] FIFO Debug Demo	Synchronous FIFO Memory *Read Pointer Underflow Bug*	**Inputs:** projects/fifo/specs/spec.md, projects/fifo/rtl/, projects/fifo/logs/sim.log. **Outcome:** Identifies read pointer decrement when FIFO is empty and generates a patch enforcing empty flag gating.
[3] ALU Plan Demo	Arithmetic Logic Unit (ALU) *Full Verification Suite Synthesis*	**Inputs:** examples/specs/alu_spec.md, examples/rtl/. **Outcome:** Synthesizes 8 functional features, 14 test cases, 12 SystemVerilog coverpoints, 10 SVA properties, and HTML dashboard.
[4] FIFO Plan Demo	Synchronous FIFO Queue *Memory & Protocol Architecture*	**Inputs:** projects/fifo/specs/spec.md, projects/fifo/rtl/. **Outcome:** Generates full testbench plan covering full/empty boundaries, simultaneous read/write bursts, and watermarking SVA assertions.

VERIFICATION TIP: The Quick Demos automatically populate all file paths and create dedicated output folders (output/demo_alu_debug/, output/demo_fifo_plan/), leaving existing work untouched.

Chapter 7: Option 4 — Clean Exit & Headless CI/CD Automation

Selecting option [4] Exit in the wizard terminates the interactive session cleanly and displays advanced command-line (CLI) usage patterns for automated nightly regression flows and CI/CD pipelines.

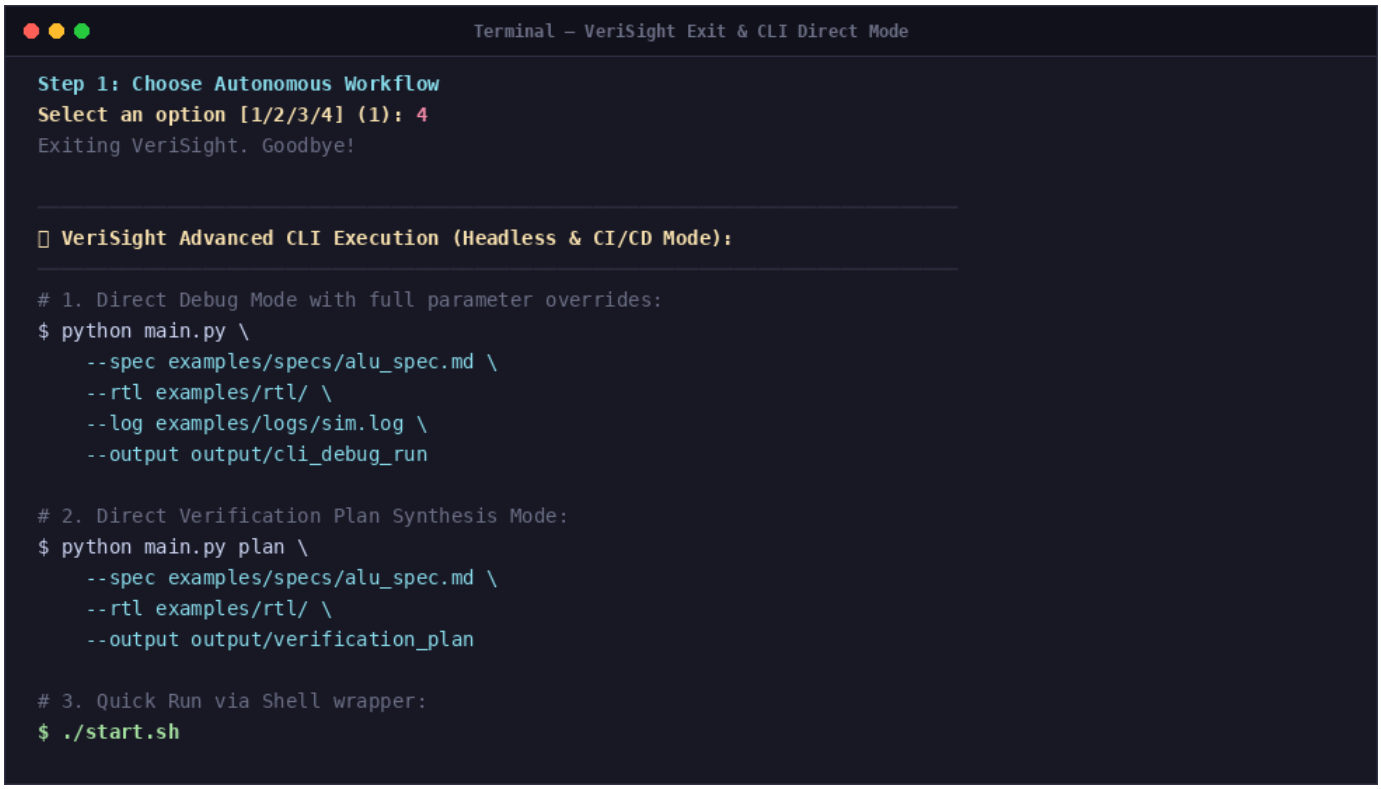


Figure 7.1: Option 4 Exit Screen — Clean Termination & Direct CLI Power Commands

Automated Headless & Regression Execution (CI/CD)

While the interactive wizard is ideal for interactive debugging, enterprise semiconductor verification requires automated execution inside LSF, Slurm, Jenkins, or GitHub Actions. VeriSight supports full headless command-line execution without requiring interactive user input:

1. Direct Simulation Debug Execution:

```
python main.py \
--spec examples/specs/alu_spec.md \
--rtl examples/rtl/ \
--tb examples/tb/ \
--log examples/logs/sim.log \
--output output/regression_run_01 \
--provider gemini \
--model gemini-2.0-flash
```

2. Direct Verification Plan Synthesis Execution:

```
python main.py plan \
--spec examples/specs/alu_spec.md \
--rtl examples/rtl/ \
--output output/verification_plan \
--provider anthropic
```

Comprehensive CLI Parameter Reference Table:

CLI Argument	Type / Default	Description & Operational Function
--spec <path>	File Path (.md, .pdf, .txt)	Path to architectural specification document.
--rtl <path>	Directory or File (.sv, .v)	Path to Design Under Test (DUT) hardware description files.
--tb <path>	Directory or File (.sv)	Path to UVM / SystemVerilog testbench source files.
--log <path>	File Path (.log, .txt)	Path to simulation log containing failure messages.
--output <dir>	Directory (default: output)	Destination directory for generated reports, HTML, and patches.
--provider <name>	gemini | anthropic	Selects cloud LLM provider (defaults to detected environment key).
--model <name>	Model String	Overrides target LLM model name (e.g. gemini-2.0-flash, claude-sonnet-5).
--no-rag	Boolean Flag	Disables ChromaDB vector knowledge base queries for air-gapped runs.
-i, --interactive	Boolean Flag	Launches the interactive wizard interface.

Chapter 8: Verification Engineering Best Practices & Sign-Off Checklist

To achieve tapeout-ready silicon sign-off, functional verification teams must adhere to disciplined engineering methodologies. This chapter outlines best practices for integrating VeriSight into production semiconductor design flows.

1. Closing the Verification Coverage Hole

A common pitfall in digital verification is relying exclusively on line and branch code coverage. While code coverage confirms that specific HDL statements executed, it provides zero assurance that functional features operate correctly under stressful corner-cases. VeriSight Agent 2 addresses this by enforcing **multi-dimensional cross coverage**:

- Cross operational modes with bus error responses (e.g. burst_type × error_resp).
- Cross FIFO occupancy thresholds with concurrent read/write transactions.
- Sample coverage points strictly synchronous to design clocks rather than asynchronous events.

2. Formal SystemVerilog Assertions (SVA) Guidelines

VeriSight synthesizes SVA properties using overlapping (|->) and non-overlapping (|=>) implication operators. Follow these guidelines when integrating dut_assertions.sv into your testbench:

- **Use bind instantiations:** Never manually edit RTL source files to insert assertions. Use SystemVerilog bind directives to connect dut_assertions.sv directly to DUT signal hierarchies.
- **Disable during reset:** Ensure all properties include active-low or active-high reset disqualifiers (e.g. disable iff (!rst_n)) to prevent false assertions during power-on initialization.
- **Pair Assert with Cover:** For every assert property, include an identical cover property to verify in simulation that the assertion precondition was actually exercised.

3. Tapeout Sign-Off Readiness Checklist (Table 8.1)

Sign-Off Milestone	Verification Requirement	VeriSight Tool Validation
Requirement Traceability	100% of specification requirements mapped to active test scenarios and covergroups.	Verify via traceability_matrix.csv. Ensure zero unmapped requirements in verification_plan.html.
Functional Coverage Closure	100% functional coverage reached across all defined coverpoints and cross bins.	Simulate with synthesized coverage_model.sv in Questa/VCS coverage engines.
Zero X-Propagation Bugs	No uninitialized states or unknown logic values propagating to memory or interfaces.	Run VeriSight X-Tracer gate-level backward propagation on gate-level simulation logs.
Clean Regression Log Analysis	Zero UVM_ERROR or UVM_FATAL banners across random seed regression runs.	Execute Agent 1 batch regression mode. Verify error.json reports 0 failures.

Chapter 9: System Schemas, Configuration & Troubleshooting

This chapter details the underlying Pydantic schema representations, key environment configuration flags in verisight/config.py, and troubleshooting resolutions for common runtime errors.

VeriSight Unified Configuration Schema (verisight/config.py)

VeriSight encapsulates all subsystem configurations into typed Pydantic models. Engineers can customize these settings programmatically or via environment variables:

```
class VeriSightConfig(BaseModel):
llm: LLMConfig = Field(default_factory=LLMConfig)
chroma: ChromaConfig = Field(default_factory=ChromaConfig)
pipeline: PipelineConfig = Field(default_factory=PipelineConfig)
xtracer: XTracerConfig = Field(default_factory=XTracerConfig)
fix: FixConfig = Field(default_factory=FixConfig)
log_level: str = 'INFO'
```

Troubleshooting & Error Remediation Guide (Table 9.1)

Observed Issue / Error	Underlying Cause	Immediate Corrective Action
API Key Not Detected LLMAuthenticationError	Missing GEMINI_API_KEY or ANTHROPIC_API_KEY in current shell.	Run the wizard and select option 3 or 4 to enter your key, or export it in your shell: export GEMINI_API_KEY='AIza...'.
Path Not Found in WSL C:\path\file.sv	Windows drive paths are located under the WSL mount point /mnt/c/.	VeriSight auto-converts C:\ to /mnt/c/. Verify the path exists in Linux via ls /mnt/c/....
Specification PDF Ingestion Fails	Scanned image PDF lacking searchable text layer or missing pypdf.	Ensure pypdf is installed (pip install pypdf). For scanned documents, run OCR prior to ingestion.
ChromaDB SQLite3 Lock	Concurrent process accessing the local vector database.	Remove temporary database locks via rm verisight_knowledge_db/*.lock or run with --no-rag flag.
Git Patch Fails to Apply patch does not apply	RTL source file was modified after the diagnostic report was generated.	Inspect output/fifo_debug/fix/corrected_snippet.sv and apply the change manually or re-run analysis.

Appendix: Operator Command Cheatsheet & Acronym Glossary

Quick-Reference Command Dictionary

Operational Goal	Command-Line Syntax
Launch Interactive Setup Wizard	./start.sh or python main.py start
Run ALU Debug 1-Click Demo	python main.py --spec examples/specs/alu_spec.md --rtl examples/rtl/ --log examples/logs/sim.log
Run FIFO Debug 1-Click Demo	python main.py --spec projects/fifo/specs/spec.md --rtl projects/fifo/rtl/ --log projects/fifo/logs/sim.log
Synthesize Verification Plan (Markdown/PDF)	python main.py plan --spec examples/specs/alu_spec.md --rtl examples/rtl/
Run in 100% Offline Mode (No Cloud LLM)	Select option 5 in LLM Setup, or run with --no-rag and omit API keys
Execute Pytest Unit Test Suite	python -m pytest tests/ -v
Apply Generated RTL Code Fix	git apply output/fifo_debug/fix/proposed.patch
View Interactive HTML Report in Browser	open output/fifo_debug/report.html or xdg-open output/verification_plan/verification_plan.html

Semiconductor Verification Acronym Glossary

Acronym	Full Standard Definition
AST	Abstract Syntax Tree — Hierarchical syntactic tree representation of HDL code.
CDC	Clock Domain Crossing — Signals transmitted between asynchronous clock regimes.
DUT	Design Under Test — The target hardware module being simulated or verified.
RCA	Root Cause Analysis — Algorithmic isolation of the primary source of a silicon defect.
RAG	Retrieval-Augmented Generation — Semantic context injection from vector databases.
SVA	SystemVerilog Assertions — Formal temporal logic properties verifying protocol rules.
UVM	Universal Verification Methodology — IEEE 1800.2 standard class library for testbenches.
X-State	Unknown Logic State — Uninitialized logic value in 4-state simulation ('X').

VeriSight™ Silicon Verification Copilot — Engineered by **Patel Kittu** (@KITTUV03).
Mirafra Technologies • All Rights Reserved • 2026